SignalGap - Proje Tasarım Notu

Backend geliştirme bitirme projesi | Node.js ve Express.js

1. Proje fikri

Bu projede klasik bir yapılacaklar listesinden biraz farklı bir konu seçtim. Yazılım ekiplerinde bazen ortada doğrudan bir hata yoktur, ancak kimsenin tam olarak bilmediği veya kontrol etmediği bir konu vardır. Bu belirsizlikleri takip edebilmek için SignalGap adında bir API hazırladım.

Örneğin, sistem düzenli olarak yedek alıyor olabilir fakat bu yedeklerin gerçekten geri yüklenip yüklenemediği bilinmiyor olabilir. Böyle bir konu kayda alınır, bir kişiye atanır ve sonuçlanana kadar takip edilir.

2. Sistemin işleyişi

Ekipten biri araştırılması gereken bir konu fark ettiğinde başlık ve açıklama yazarak yeni kayıt açar. Sorumlu kişiyi, iş önceliğini ve risk seviyesini belirler. Çalışma başlayınca kayıt durumu değiştirilir. Konu sonuçlandığında kısa bir açıklama eklenebilir.

Kayıtların durumu discovered (fark edildi), investigating (araştırılıyor), verified (doğrulandı), resolved (çözüldü) veya accepted-risk (risk kabul edildi) olabilir. Böylece her konunun hangi aşamada olduğu anlaşılır.

3. Temel özellikler

- Yeni kayıt oluşturma, tüm kayıtları listeleme ve ID ile tek kayıt görüntüleme.
- Kaydın bilgilerini güncelleme ve gerektiğinde silme.
- Sorumlu kişi, durum, iş önceliği ve risk seviyesini tutma.
- Gelen HTTP isteklerinin yöntemi, adresi ve zamanını terminale yazan logger middleware.

Bunlara ek olarak arama, filtreleme, sayfalama, alan doğrulama ve basit rapor servisleri de bulunuyor.

4. Veri modeli

İlk sürümde veriler ayrı bir veritabanına değil, src/data/gaps.json dosyasına kaydediliyor. Böylece proje kurulumunu basit tutup API işlemlerine odaklanabildim.

Alan	Açıklama
id	Her kaydın numarası
title, description	Konu başlığı ve açıklaması
assignee, category	Sorumlu kişi ve ilgili alan
priority, riskLevel	Öncelik ve risk düzeyi (low / medium / high)
status, resolution	Kaydın aşaması ve sonuç notu
createdAt, updatedAt	Oluşturulma ve son güncellenme zamanı

5. API uçları

API, gelen istekleri HTTP yöntemine göre karşılıyor. Ana işlemler şu adresler üzerinden yapılıyor:

Yöntem	Adres	İşlem
POST	/gaps	Yeni kayıt ekleme
GET	/gaps	Kayıtları listeleme
GET	/gaps/:id	Kayıt detayı
PUT	/gaps/:id	Kayıt güncelleme
DELETE	/gaps/:id	Kayıt silme

6. Kod yapısı ve test

routes klasörü API adreslerini, controllers klasörü istek ve yanıt işlemlerini, services klasörü kayıtlarla ilgili kuralları içeriyor. JSON okuma-yazma işlemleri utils altında tutuluyor. Logger ve veri kontrolü için middleware kullanılıyor.

Temel CRUD isteklerini Postman üzerinden denedim. Kayıt oluşturma için 201, listeleme ve detay için 200, güncelleme için 200, silme için 204 yanıtı alındı. Test ekran görüntüleri teslim dosyalarına eklenecek.

7. Bu sürümün sınırları

Projede kullanıcı girişi, web arayüzü veya ayrı bir veritabanı bulunmuyor. Bu sürümde hedef, REST API yapısını, CRUD işlemlerini ve middleware kullanımını düzgün şekilde uygulamak.